

# Project 1 - Advanced RAG Platform

PRD + Architecture Roadmap | V1 -> V2 -> V3

## 1. Project Objective

Build a production-oriented, evidence-grounded RAG platform for heterogeneous corporate knowledge. It must ingest data you can realistically obtain: public company webpages/PDFs plus clearly labelled synthetic enterprise documents. The system should retrieve evidence, answer with citations, verify claims, and abstain when evidence is insufficient.

**Design principle from the supplied Level Up Coding article: 'retrieve, constrain, verify, abstain'. The article describes cleaning/chunking, dense + BM25 indexing, RRF, reranking, routing/decomposition and claim verification before answering or abstaining.**

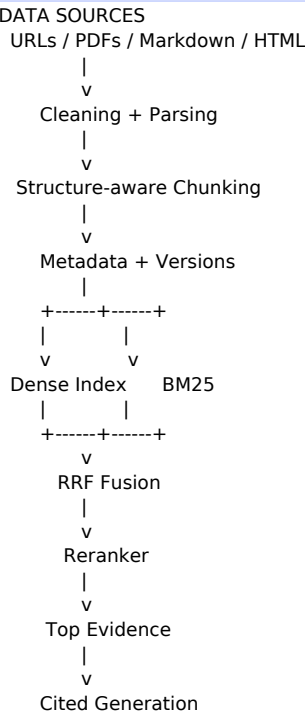
## 2. Data Strategy - Solve This First

Do not make Jira, Slack, Confluence or internal company databases prerequisites. Instead create a reproducible corpus from public corporate sources and a small, high-quality synthetic enterprise corpus. Synthetic documents should be explicitly marked synthetic.

| Source                    | Examples                                                 | Why                                                       |
|---------------------------|----------------------------------------------------------|-----------------------------------------------------------|
| Public URLs               | Policies, help centres, docs, terms, security pages      | Real corporate content and URL RAG                        |
| Public PDFs               | Reports, policies, product/security documents            | Long-form ingestion                                       |
| Synthetic enterprise docs | Policies, incidents, runbooks, ADRs, FAQs                | Lets you create versions and cross-document relationships |
| Golden evaluation set     | Questions, expected answers, evidence IDs, answerability | Lets you measure improvements                             |

## 3. V1 - Strong Baseline RAG

**Build V1 completely before adding agentic loops. The goal is a reliable baseline whose quality can be measured.**



## V1 build order

| Order | Build                             | Why                                                                 |
|-------|-----------------------------------|---------------------------------------------------------------------|
| 1     | Corpus + ingestion                | Retrieval quality is meaningless without reliable source data.      |
| 2     | Cleaning + structure-aware chunks | Chunk quality strongly affects retrieval.                           |
| 3     | Metadata + stable IDs             | Needed for citations, filtering and future version-aware retrieval. |
| 4     | Dense retrieval                   | Semantic baseline.                                                  |
| 5     | BM25                              | Handles exact policy names, IDs and terminology.                    |
| 6     | RRF                               | Combines dense and lexical rankings.                                |
| 7     | Reranker                          | Improves precision of final context.                                |
| 8     | Citations + answer generation     | Makes output inspectable.                                           |
| 9     | Golden evaluation set             | Turns tuning into measurable engineering.                           |

## V1 metrics

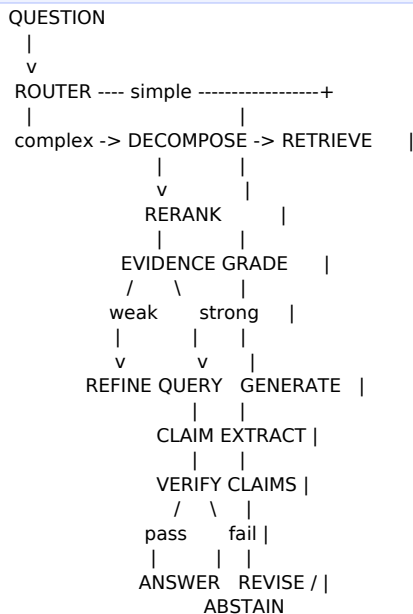
- Recall@K / Precision@K / MRR or NDCG for retrieval.
- Answer correctness and faithfulness on labelled questions.
- Citation support rate.
- Latency, token usage and retrieval/reranking time.

## V1 acceptance

- Every answer has traceable evidence.
- Dense-only vs hybrid retrieval is benchmarked.
- Answerable and unanswerable questions are included.
- All documents have stable provenance metadata.

## 4. V2 - Retrieve, Constrain, Verify, Abstain

**V2 adds the quality-control loop. It should detect weak retrieval and unsupported claims instead of blindly trusting the generator.**



### V2 additions

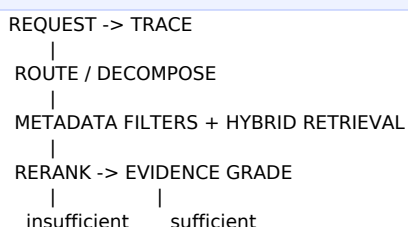
| Capability                | Why                                                    |
|---------------------------|--------------------------------------------------------|
| Query routing             | Avoid expensive reasoning for simple questions.        |
| Query decomposition       | Handles multi-part and multi-hop questions explicitly. |
| Retrieval grading         | Detects weak evidence before generation.               |
| Bounded refinement        | Retries weak retrieval with a better query.            |
| Claim extraction          | Makes verification granular.                           |
| Faithfulness verification | Checks whether claims are actually supported.          |
| Abstention                | Makes insufficient evidence a valid outcome.           |
| Weakest-claim reporting   | Avoids hiding one bad claim behind an average score.   |

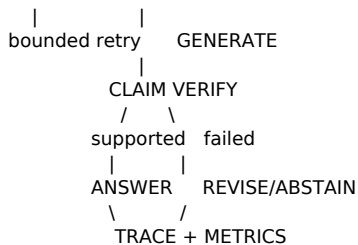
### V2 acceptance

- Complex questions produce traceable sub-queries.
- Weak retrieval triggers a bounded retry.
- Unsupported claims are revised, removed or cause abstention.
- Verification decisions are visible in traces.

## 5. V3 - Production-oriented RAG

**V3 turns the prototype into a portfolio-grade system: temporal correctness, evaluation infrastructure, observability, freshness and controlled scale.**





| Capability              | What to add                               | Why                                                    |
|-------------------------|-------------------------------------------|--------------------------------------------------------|
| Version-aware retrieval | effective_date, version, supersedes links | Avoid obsolete policy answers.                         |
| Freshness               | scheduled recrawl + content hashes        | Keep URL RAG current.                                  |
| Evaluation harness      | Golden set + regression + ablations       | Prove each improvement.                                |
| Observability           | End-to-end traces                         | Debug quality/latency failures.                        |
| Cost/latency            | Per-stage telemetry                       | Make model/retrieval trade-offs explicit.              |
| Provider abstraction    | Pluggable LLM/embedding/reranker          | Experiment without rewrites.                           |
| Scale benchmark         | Optional 1M+ chunks                       | Demonstrate systems engineering without fake 10M data. |

## 6. Data Model

Document: id, source\_url, type, title, version, effective\_date, fetched\_at, content\_hash, supersedes, metadata

Chunk: id, document\_id, section\_path, contextual\_prefix, text, token\_count, embedding\_id, sparse\_terms

EvaluationCase: question, expected\_answer, relevant\_doc\_ids, relevant\_chunk\_ids, answerable, difficulty, tags

## 7. Suggested Stack

| Layer         | Choice                                   | Reason                                    |
|---------------|------------------------------------------|-------------------------------------------|
| Python        | Python                                   | Fits Applied AI + DE background.          |
| Orchestration | LangGraph from V2                        | Useful once loops and verification exist. |
| Vector        | Qdrant or LanceDB                        | Filtering / local scale experiments.      |
| Sparse        | BM25                                     | Exact lexical matching.                   |
| Reranker      | Cross-encoder or API                     | Final evidence precision.                 |
| Evaluation    | Python harness + optional Ragas/DeepEval | Keep metrics transparent.                 |
| Observability | OpenTelemetry + trace tooling            | Trace retrieval and generation.           |
| Cloud         | GCP + Docker                             | Uses existing cloud skills.               |
| Pipeline      | Airflow for scheduled ingestion          | Uses DE experience where justified.       |

## 8. Do NOT Build First

- No multi-agent system at the beginning.
- No 10M fake documents.
- No knowledge graph before measurable retrieval.
- No latency optimization before quality baselines.

- No 'near-zero hallucination' claim without your own benchmark.

## 9. Build Sequence

M1 Corpus + ingestion  
M2 Cleaning + chunking + metadata  
M3 Dense + BM25 + RRF  
M4 Reranker + citations + baseline eval -> V1  
M5 Router + decomposition  
M6 Retrieval grading + refinement  
M7 Claim extraction + verification + abstention -> V2  
M8 Version/freshness + evaluation harness  
M9 Observability + cost/latency + regression  
M10 Scale benchmark + polish -> V3

## 10. Portfolio Deliverables

- Architecture + PRD.
- Reproducible ingestion CLI/config.
- Public + synthetic corpus with provenance.
- Golden evaluation dataset.
- V1/V2/V3 GitHub releases.
- Evaluation dashboard and failure analysis.
- README explaining trade-offs.
- Demo showing evidence, citations, verification and abstention.

## 11. Source Basis

Source basis: the supplied June 15, 2026 Level Up Coding article presents a pipeline that gets data, cleans/chunks it, builds dense and BM25 indexes, fuses rankings with RRF, reranks candidates, routes/decomposes questions, and aims for cited answers or calibrated abstention. This project adapts those ideas to a corpus you can actually create and evaluate.